

Guía de Proyecto v2.0

Valdris: El Núcleo Profundo

Juego por Turnos · JavaFX · Estructuras de Datos Propias

Asignatura	Metodología de la Programación + Estructuras de Datos
Grupo	H12GEXTRA — Marcos Castro · Ventura Pacheco · Marino Rodríguez
Versión	v2.0 — Incluye narrativa, personajes, zonas y diseño completo
Lenguaje	Java 21 · JavaFX · Gson 2.10.1 · IntelliJ IDEA

Qué contiene esta versión
• v1.0: arquitectura, clases, roadmap, advertencias técnicas, instrucciones para IA. • v2.0 añade: historia completa de Valdris, tres personajes jugables con backstory y stats, • cinco zonas con ambientación y enemigos propios, diseño del boss final Malachar, • mecánicas narrativas (llaves, acertijos, pasadizos, escaleras) y su implementación técnica.

1. El Mundo — Valdris

Valdris es un continente de fantasía clásica dividido en cinco zonas con ecosistemas, criaturas y ambientaciones distintas. Hace trescientos años, los cinco reinos de Valdris unieron fuerzas para sellar a Malachar — un archimago que había descubierto cómo absorber la esencia vital de las criaturas mágicas para volverse inmortal — dentro de una fortaleza subterránea construida bajo el corazón del continente: El Núcleo Profundo.

Cinco guardianes, uno por reino, fueron encargados de mantener el sello activo con su propia vida. Trescientos años después, los guardianes han muerto de viejos. El sello se debilita. Las criaturas mágicas de cada zona empiezan a enloquecerse porque algo las está absorbiendo desde abajo. Los cinco reinos no se ponen de acuerdo en qué hacer. Tres personas deciden actuar por su cuenta.

El giro central — Malachar y la verdad

La verdad que los reinos ocultaron

- Malachar no buscaba el poder por ambición. Descubrió que Valdris tiene un parásito: una entidad exterior al mundo que se alimenta lentamente de toda vida mágica.
- Su cálculo: en dos o tres generaciones el parásito lo habría consumido todo.
- Su método: concentrar toda la magia del mundo en un ser inmortal que pudiera resistir al parásito.
- El problema: hacerlo mataba a miles de criaturas inocentes. Los reinos lo sellaron sin escucharle.
- Ahora el parásito sigue ahí. Más cerca que nunca. Y el sello era lo único que lo frenaba.

"¿Veis lo que está pasando ahí fuera? Eso es el parásito. Y vosotros acabáis de romper lo único que lo frenaba."

El jugador llega al final esperando matar a un monstruo y encuentra a un anciano agotado que lleva trescientos años solo y atrapado. La decisión final no es matar al villano. Es decidir qué hacer con la verdad. Eso da peso moral real sin necesitar mecánicas complejas adicionales.

Las tres decisiones finales

Decisión	Consecuencia narrativa	Estado GameState
Terminar lo que Malachar empezó	El jugador absorbe el parásito junto a Malachar. Valdris sobrevive. El precio es desconocido.	ENDING_SACRIFICE
Destruir el sello y liberar a Malachar	Malachar queda libre. Promete combatir al parásito. Los reinos lo perseguirán. El mundo elige su destino.	ENDING_FREE
Sellar el Núcleo Profundo de nuevo	El parásito sigue creciendo. Pero hay tiempo. Los reinos ahora tienen la verdad y deben decidir.	ENDING_SEAL

2. Los Tres Personajes Jugables

Los tres personajes comparten el mismo objetivo pero llegan a él desde ángulos distintos. Sus stats reflejan su historia. El jugador elige uno al inicio y juega toda la partida con él. Los otros dos aparecen mencionados en el lore de las zonas — existen en el mundo aunque no sean el personaje elegido.

Kael

El Portador de la Llama Rota

Aprendiz del último guardián del sello del reino de Embrath. Cuando su maestro murió, Kael intentó asumir el rol pero no tenía el linaje necesario — el sello lo rechazó y le quemó la mano derecha hasta el hueso. Ahora lleva un guantelete de hierro sobre esa mano y una deuda que no sabe cómo saldar. Va al Núcleo Profundo porque es lo único que le quedaba que hacer. No es exactamente un guerrero: es un espadachín con magia residual quemada en el cuerpo. Sus ataques cuerpo a cuerpo pueden activar descargas de energía ígnea que no controla del todo.

Stat	Valor	Nota
HP	110	Resistente pero no un tanque
Daño base	18	El más alto de los tres
Mov. points	3	Movilidad media
Rango ataque	1	Solo cuerpo a cuerpo
	Descarga Ígnea	30% de probabilidad de infligir quemadura al atacar (daño extra siguiente turno)

Syra

La Voz Sin Eco

Elfa de los bosques de Lireth cuyo clan fue el primero en notar el enloquecimiento de los animales mágicos. Pasó dos años rastreando el origen antes de entender que venía de abajo. Es la más informada de los tres: tiene mapas parciales del Núcleo Profundo y sabe qué esperar. Lo que no tiene es fuerza para hacerlo sola. Va porque es la única que sabe realmente a qué se enfrentan, aunque no se lo ha dicho a nadie todavía. No es exactamente una arquera: es una rastreadora que usa flechas encantadas cargadas con efectos según los materiales recogidos en cada zona.

Stat	Valor	Nota
HP	75	La más frágil de los tres
Daño base	12	Bajo sin encantamiento activo
Mov. points	5	La más rápida del mapa
Rango ataque	3	Ataca desde lejos
	Flecha Encantada	El efecto de ataque varía según el ítem equipado: fuego, hielo, parálisis...

Dorath

El Excomulgado

El mejor clérigo de la Orden de los Cinco Sellos — la orden religiosa creada para venerar a los guardianes. Lo excomulgaron cuando investigó los textos originales del sellado y encontró inconsistencias: la Orden sabía desde el principio que el sello era temporal. Va al Núcleo Profundo porque quiere la verdad, y porque la Orden le quitó todo lo demás. No es exactamente un mago: es un clérigo caído con acceso a magia sagrada y oscura a la vez. Puede curar, puede maldecir, y tiene el mayor rango de ataque. Lento, pero el más versátil.

Stat	Valor	Nota
HP	80	Frágil en combate directo
Daño base	14	Mágico, ignora armadura física
Mov. points	2	El más lento de los tres
Rango ataque	4	Mayor rango de ataque
	Palabra Dual	Alterna entre cura (turno impar) y maldición al enemigo (turno par) automáticamente

3. Las Cinco Zonas de Valdris

El mapa se divide en cinco zonas que el jugador recorre en orden. Cada zona tiene su propia ambientación, enemigos temáticos, un acertijo o mecánica de bloqueo obligatoria para avanzar, y al menos un ítem de zona único que solo se obtiene ahí. Las zonas 1-4 culminan con un enemigo élite (mini-boss) que suelta el ítem necesario para entrar a la siguiente zona.

Zona 1 — Los Campos Grises	
Llanuras muertas y aldeas abandonadas. Primera zona, tutorial implícito. La tierra está reseca, los pozos envenenados. Los aldeanos que no huyeron han sido corrompidos por la absorción mágica del parásito y atacan sin reconocer a nadie.	
Enemigos	Aldeanos corrompidos (Warrior), Lobos enloquecidos (Archer rápido), Espectro de campo (Mage débil)
Acertijo	Puente roto sobre el río gris: cuatro palancas en las orillas, solo una secuencia correcta lo reconstruye. Secuencia errónea → trampa de flechas.
Mini-boss	El Alcalde Corrompido — Warrior de alto HP que al morir suelta la Llave de Hierro.
Item de zona	Llave de Hierro — abre la puerta norte hacia el Bosque de Lireth.
Pasadizo secreto	Detrás del molino: celda de tipo DOOR_HIDDEN que se activa al empujar una piedra específica. Lleva a un almacén con Poción de Fuerza.

Zona 2 — El Bosque de Lireth	
Bosque mágico antiguo en proceso de corrupción. Los árboles crecen retorcidos hacia abajo, la luz es verde y enfermiza. Los espíritus del bosque que antes protegían a los elfos ahora los atacan. Syra conoce este lugar — si es el personaje elegido, hay diálogos adicionales.	
Enemigos	Espíritus de árbol (Warrior lento, golpe alto), Arqueros elfos poseídos (Archer preciso), Guardián de raíces (Mage que inmoviliza celdas)
Acertijo	Laberinto vegetal: raíces bloquean rutas. Hay que cortar las raíces en el orden correcto — el orden incorrecto hace crecer más raíces y bloquea retroceso.
Mini-boss	El Espíritu Madre — Mage de alto rango que invoca refuerzos. Al morir suelta la Semilla Resonante.
Item de zona	Semilla Resonante — resuena cerca de trampas ocultas en zonas posteriores, actúa como detector pasivo.
Escalera	Acceso a una planta subterránea del bosque (raíces profundas) con cofre oculto y enemigo extra.

Zona 3 — Las Minas de Karath	
Minas enanas excavadas hace siglos, ahora abandonadas. Varios pisos conectados por escaleras y ascensores de vagoneta. La piedra misma está viva — los gólems emergieron solos cuando la absorción mágica despertó los minerales encantados que los enanos dejaron atrás.	
Enemigos	Gólems de piedra (Warrior muy lento, altísima defensa), Murciélagos de cristal (Archer rápido, frágil), Enano espectral (Mage con ataque de área)
Acertijo	Mecanismo de vagoneta en tres niveles: activar palancas en pisos distintos en el orden correcto para abrir la compuerta al piso inferior. Orden incorrecto → vagoneta aplasta celda bloqueando el paso temporalmente.
Mini-boss	El Gólem Maestro — Warrior que ocupa 2x2 celdas. Al morir suelta el Fragmento de Sello.
Item de zona	Fragmento de Sello — necesario para interactuar con los mecanismos de cierre del Núcleo Profundo en el final.
Escaleras	Tres pisos distintos. Piso -1 (entrada), Piso -2 (minas activas), Piso -3 (cámaras profundas con el mini-boss).

Zona 4 — La Torre de Embrath	
Torre mágica en ruinas donde entrenaban los guardianes del sello. Kael conoce este lugar — si es el personaje elegido, hay texto adicional al entrar. Los constructos mágicos que guardaban la torre siguen activos con sus últimas órdenes: no dejar salir a nadie.	
Enemigos	Constructos de energía (Warrior rápido), Guardianes espectrales (Archer de largo rango), Archivero corrompido (Mage que debilita stats)
Acertijo	Suelo de runas: baldosas con símbolos que hay que pisar en el orden del antiguo juramento de los guardianes. Descifrable con un pergamino encontrado en Zona 1. Orden incorrecto → daño a toda la habitación.
Mini-boss	El Guardián Sin Nombre — espectro del último guardián, el maestro de Kael. Al morir suelta la Esencia Ígnea.
Item de zona	Esencia Ígnea — potencia los ataques de Kael, desbloquea ruta alternativa en el Núcleo Profundo.
Pasadizo secreto	Tras la biblioteca de la torre: pasadizo que conecta directamente con la entrada del Núcleo Profundo, saltando parte del descenso.

Zona 5 — El Núcleo Profundo	
La fortaleza subterránea donde fue sellado Malachar. No es una mazmorra de combate normal: es un lugar que respira, que recuerda, que ha estado esperando. Las sombras aquí tienen forma propia. Y en el centro hay algo que lleva trescientos años solo.	
Enemigos	Sombras absorbidas (Warrior sin HP máximo fijo — escalan con el parásito), Ecos de magia (Archer que copia el último ataque del jugador), El Filtro (Mage jefe de sala que bloquea el paso al núcleo)
Sin acertijo clásico	La última sala no tiene puzzle de mecánica. Tiene una conversación. Malachar habla. El jugador decide.
Boss — Malachar	Dos fases: Fase 1 (>50% HP): Mage de altísimo rango, ataca con el parásito mismo. Fase 2 (<50% HP): para de atacar. Empieza a hablar. El combate se detiene. Aparece el menú de decisión final.
Los tres finales	ENDING_SACRIFICE / ENDING_FREE / ENDING_SEAL (ver Sección 1).
Item especial	Los tres fragmentos recogidos en zonas anteriores (Llave, Semilla, Fragmento de Sello, Esencia) desbloquean diálogos adicionales con Malachar si están en el inventario.

4. Mecánicas Especiales y su Implementación Técnica

Mecánica	Implementación en código
Llaves y puertas	Cell de tipo DOOR_LOCKED. Al intentar entrar: TurnManager comprueba <code>Player.inventory.contains(keyItem)</code> . Si no tiene la llave → mensaje de bloqueo. Si tiene → <code>Cell.type = DOOR</code> (desbloqueada).
Pasadizos secretos	Arista oculta en el Grafo. La celda trigger es FLOOR normal. Al pisar: <code>Room.checkSecretTrigger(row,col)</code> devuelve true → <code>Dungeon.activateHiddenPassage(roomId)</code> añade la Arista al Grafo.
Escaleras (pisos)	NodoGrafo especial con id que contiene 'stairs_up' o 'stairs_down'. La Arista que lo conecta tiene dirección 'arriba' o 'abajo'. <code>Dungeon.getAdjacentRooms()</code> las incluye normalmente.
Enemigo suelta item	Enemy tiene campo <code>droplItem</code> : Item (puede ser null). En <code>Enemy.onDeath()</code> : si <code>droplItem != null</code> → <code>room.placeItemOnCell(row, col, droplItem)</code> . El item queda en la celda hasta que el jugador la pisa.
Acertijos de palancas	Room tiene <code>LSE leverCells</code> y <code>int[] correctSequence</code> . <code>LeverManager.activate(cell)</code> registra el orden. Al completar: <code>LeverManager.checkSequence()</code> compara con <code>correctSequence</code> → abre puerta o activa trampa.
Acertijo de runas	Igual que palancas pero las Cell son de tipo RUNE. El jugador las activa al pisarlas. La secuencia correcta se obtiene de un pergamino (SpecialItem con <code>effectData = 'rune_sequence:3,1,4,2'</code>).
Items de zona con efecto pasivo	Item implementa interfaz <code>PassiveEffect</code> con método <code>onRoomEnter(Player)</code> . TurnManager llama a este método al cambiar de sala. La Semilla Resonante activa <code>Cell.highlight</code> en celdas trampa cercanas.
Fases del boss	Malachar extiende Enemy con campo <code>int phase = 1</code> . En <code>executeTurn()</code> : si <code>hp < maxHp/2</code> y <code>phase == 1</code> → <code>phase = 2</code> , <code>TurnManager.triggerDialogue('malachar_phase2')</code> . En fase 2 deja de atacar.
Decisión final	<code>TurnManager.triggerFinalChoice()</code> muestra tres opciones al Controller JavaFX. La opción elegida llama a <code>GameState.setEnding(ENDING_X)</code> y <code>TurnManager.endGame()</code> .
Coste de movimiento variable	Cell tiene <code>int moveCost = 1</code> por defecto. BFSMovimiento resta <code>cell.moveCost</code> en lugar de 1 fijo. Terreno especial (agua=2, trampa=3) simplemente cambia ese valor al generar la Room.

5. Arquitectura y Estructura de Clases

Resumen de las decisiones de arquitectura tomadas en v1.0. Ver documento anterior para detalle completo.

Capas del sistema

Capa	Paquete	Responsabilidad
Estructuras base	MisEstructurasDeDatos	LSE, Cola, Grafo, ABB — ya implementadas, no tocar
Modelo / Dominio	juego.modelo	Cell, Room, Dungeon, Unit, Player, Enemy, Item, GameState
Lógica del juego	juego.logica	TurnManager, BFSMovimiento, LeverManager, LectorJSON
Presentación JavaFX	juego.vista	Controllers, GridPane de habitación, HUD, menús

Clases clave y sus relaciones

Clase	Extiende / Implementa	Atributos clave añadidos en v2
CellType (enum)	—	FLOOR, WALL, DOOR, DOOR_LOCKED, DOOR_HIDDEN, LEVER, RUNE, STAIRS_UP, STAIRS_DOWN
Cell	—	type, unit, item, moveCost, isRevealed, isHighlighted
Room	InterfazDatosNodo	id, name, cells[], leverCells: LSE, correctSequence: int[]
Dungeon	—	grafo: Grafo, currentRoom, floorLevel: int
Unit (abstract)	Movable, Attackable	name, hp, maxHp, row, col, roomId: String (no Room directa)
Player	Unit	movePoints, inventory: LSE, equippedWeapon, equippedArmor, characterType: enum
Enemy (abstract)	Unit	attackRange, damage, dropItem: Item, phase: int
Warrior/Archer/Mage	Enemy	executeTurn() con IA propia según tipo
Item (abstract)	Usable	name, description, use(Unit target)
Weapon/Armor/Potion /SpecialItem	Item	atributos específicos + effectData: String para items pasivos
TurnManager	—	turnQueue: Cola, log: LSE, gameState: GameState
GameState	—	player, dungeonData (plano), turnNumber, ending: EndingType enum

6. Roadmap de Desarrollo

Fase 0 — Diseño previo — COMPLETADO

- UML Capa 1 (herencia) y Capa 2 (atributos) generados.
- Historia, personajes y zonas definidos (este documento).
- Checklist de requisitos del enunciado validado.
- Advertencias técnicas críticas identificadas.

Fase 1 — Núcleo de datos — EMPEZAR AQUÍ

- CellType (enum con todos los tipos incluyendo STAIRS, LEVER, RUNE, DOOR_LOCKED, DOOR_HIDDEN).
- Cell: type, unit, item, moveCost, isRevealed, isHighlighted.
- Room: Cell[], id, name, leverCells, correctSequence, getCell, setUnit, placeItemOnCell.
- Room implements InterfazDatosNodo (getNombre=id, getTipo='room').
- Dungeon: Grafo, addRoom, connect, activateHiddenPassage, floorLevel.
- Unit (abstracta) con roomId String, Player con inventory LSE y characterType.
- Enemy (abstracta) con dropltem y phase, Warrior/Archer/Mage concretas.
- Item (abstracta), Weapon, Armor, Potion, SpecialItem con effectData.
- TurnManager: Cola, LSE log, nextTurn, addLog.
- GameState con EndingType enum y estructura plana para Gson.
- → Tests JUnit para todas las clases del modelo antes de continuar.

Fase 2 — Lógica del juego

- BFSMovimiento.getCellsInRange() sobre Cell[] usando moveCost variable.
- Player.move(r,c): consume movePoints, detecta STAIRS/DOOR/DOOR_LOCKED.
- LeverManager: registra secuencia de palancas, compara con correctSequence.
- Room.checkSecretTrigger(r,c): activa pasadizos ocultos en el Dungeon.
- Enemy.executeTurn() por tipo: Warrior persigue, Archer mantiene rango, Mage optimiza área.
- Sistema de combate: attack, takeDamage, onDeath con dropltem.
- Sistema de inventario: add, remove, use, equip.
- TurnManager.triggerDialogue y triggerFinalChoice para el boss final.
- → Tests JUnit para BFS, palancas, combate, turnos y decisión final.

Fase 3 — Persistencia JSON

- GameState serializable: estructura plana con List y List.
- LectorJSON.guardar(GameState, ruta) y LectorJSON.cargar(ruta) → reconstruye Dungeon.
- TurnManager.reconstruirReferencias(): reestablece Unit.room desde roomId tras carga.
- Manejo de excepciones: IOException, JsonSyntaxException, InvalidStateException propia.
- JSON de ejemplo con partida guardada para entregar con el proyecto.
- → Tests JUnit para guardar/cargar, casos de error y reconstrucción de referencias.

Fase 4 — Interfaz JavaFX

- GridPane dinámico que dibuja Cell[][] con colores por tipo (FLOOR, WALL, DOOR...).
- Resaltado de celdas alcanzables (BFSMovimiento result) al seleccionar jugador.
- Indicadores de LEVER, RUNE, STAIRS en la celda correspondiente.
- Panel de selección de personaje al inicio (Kael / Syra / Dorath con stats).
- Panel lateral: HP, movePoints, inventario, turno actual, log de acciones.
- Diálogos de Malachar en el boss final y menú de decisión final.
- Pantallas de fin de partida según EndingType.
- → Pruebas manuales. No hay tests JUnit para la vista.

7. Advertencias y Decisiones Críticas

ADVERTENCIA 1 — Serialización del Grafo con Gson

- Gson no puede serializar Grafo directamente por referencias circulares.
- Solución: GameState guarda List y List (estructura plana).
- LectorJSON reconstruye el Dungeon desde esas listas al cargar.
- NO implementar JsonSerializer custom sin consultarlo primero.

ADVERTENCIA 2 — Referencias circulares Unit ↔ Room

- Unit NO guarda Room room. Guarda String roomId.
- Al cargar: TurnManager.reconstruirReferencias() reestablece la referencia buscando por id.
- Aplicar desde el primer día — cambiarlo después implica refactorizar Player, Enemy y TurnManager.

ADVERTENCIA 3 — CellType debe incluir todos los tipos desde el inicio

- Añadir STAIRS_UP, STAIRS_DOWN, LEVER, RUNE, DOOR_LOCKED, DOOR_HIDDEN desde el principio.
- Añadirlos después obliga a modificar BFSMovimiento, TurnManager y el renderer JavaFX simultáneamente.
- Es un enum: añadir un valor nuevo es trivial ahora, costoso después si hay switches por todas partes.

ADVERTENCIA 4 — No empezar por JavaFX

- La Fase 4 no empieza hasta que Fases 1, 2 y 3 pasan todos sus tests.
- La UI es un observador pasivo del modelo — no debe contener ningún cálculo de juego.
- Error frecuente: descubrir fallos de arquitectura cuando ya hay código de vista mezclado con lógica.

ADVERTENCIA 5 — BFS y movimiento son dos operaciones distintas

- BFSMovimiento.getCellsInRange() calcula celdas alcanzables. NO mueve al jugador.
- Player.move(r,c) ejecuta el movimiento real. Se llama solo cuando el jugador confirma destino.
- Si los mezclas no podrás mostrar el resaltado en JavaFX sin ejecutar el movimiento.

8. Instrucciones para la IA en Conversaciones Futuras

Contexto del proyecto

- Juego por turnos JavaFX en Java 21. Historia: Valdris, Malachar, El Núcleo Profundo.
- Tres personajes: Kael (espadachín ígnea), Syra (rastreadora élfica), Dorath (clérigo caído).
- Cinco zonas con enemigos, acertijos y mecánicas propias. Boss final con decisión moral.
- Código base existente: Grafo, LSE, Cola, Pila, ABB — funcionando, no reimplementar.
- Restricción: NO usar `java.util.*` para estructuras de datos del juego.

Reglas de comportamiento para la IA

- Cuando el alumno pide implementar una clase: escribe el código completo, no fragmentos.
- Siempre verificar que el código usa LSE/Cola/Grafo propias, no `java.util.*`.
- Si el alumno propone algo que contradice una advertencia de la Sección 7, decirlo antes de continuar.
- No repetir lo que ya está decidido. Asumir que el alumno conoce este documento.
- Dar recomendación directa con justificación cuando haya varias opciones.
- El orden de implementación recomendado está en la Sección 6 Fase 1.

Cosas que la IA NO debe hacer

- Sugerir `ArrayList`, `HashMap` o cualquier `java.util` para lógica del juego.
- Proponer empezar JavaFX antes de que la lógica esté testeada.
- Añadir `Unit.room` (referencia directa) en lugar de `Unit.roomId` (String).
- Implementar `JsonSerializer` custom de Gson sin que el alumno lo solicite.
- Mezclar lógica de juego en los controllers de JavaFX.
- Dar la razón por defecto si el alumno contradice las decisiones de diseño de este documento.